PROJECT ON STOCK MARKET DATA ANALYSIS

OBJECTIVE

Identify the market or industry you want to analyze. Define the specific objectives of your analysis (e.g., predicting market growth, understanding consumer behavior, etc.).

DATA COLLECTION

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv('stocks.csv')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 248 entries, 0 to 247
Data columns (total 8 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Ticker     248 non-null    object
 1   Date       248 non-null    object
 2   Open       248 non-null    float64
 3   High       248 non-null    float64
 4   Low        248 non-null    float64
 5   Close      248 non-null    float64
 6   Adj Close  248 non-null    float64
 7   Volume     248 non-null    int64
dtypes: float64(5), int64(1), object(2)
memory usage: 15.6+ KB
```

```
df.head()
```

	Ticker	Date	Open	High	Low	Close	Adj Close	Volume
0	AAPL	2023-02-07	150.639999	155.229996	150.639999	154.649994	154.414230	83322600
1	AAPL	2023-02-08	153.880005	154.580002	151.169998	151.919998	151.688400	64120100
2	AAPL	2023-02-09	153.779999	154.330002	150.419998	150.869995	150.639999	56007100
3	AAPL	2023-02-10	149.460007	151.339996	149.220001	151.009995	151.009995	57450700
4	AAPL	2023-02-13	150.949997	154.259995	150.919998	153.850006	153.850006	62199000

EDA

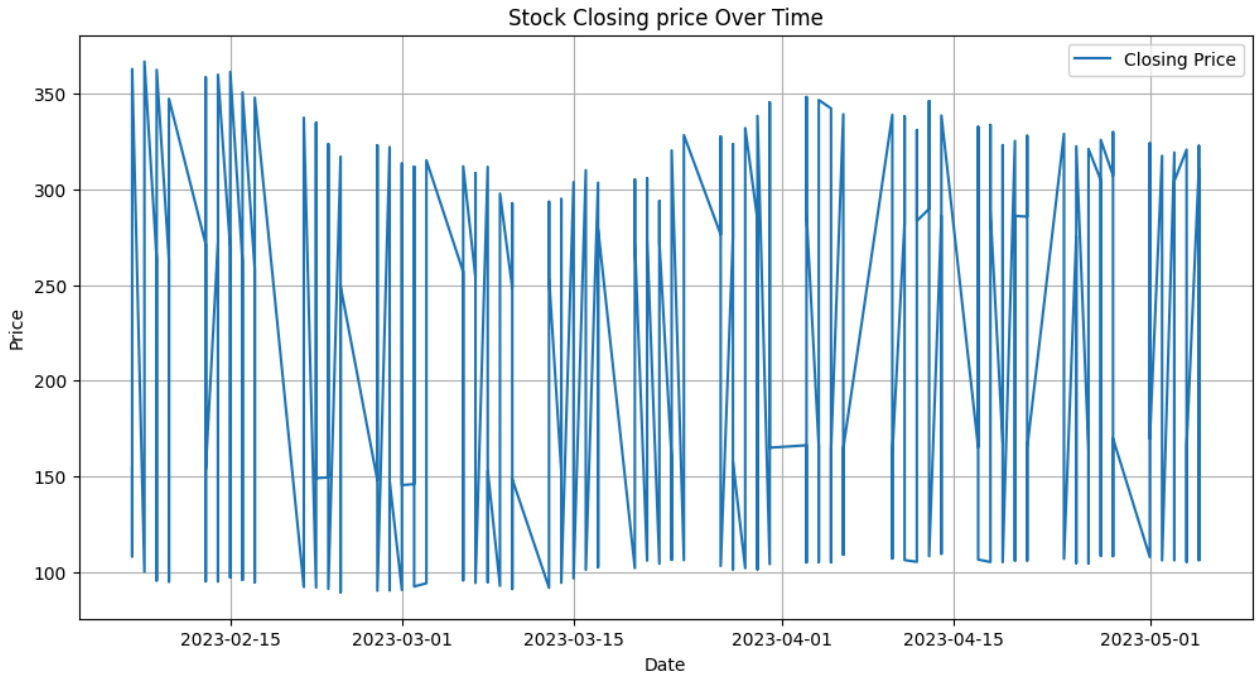
```
df.isnull().sum()
```

	0
Ticker	0
Date	0
Open	0
High	0
Low	0
Close	0
Adj Close	0
Volume	0

dtype: int64

```
df[ 'Date'] = pd .to_datetime(df[ 'Date' ])
df = df.sort_values('Date')
```

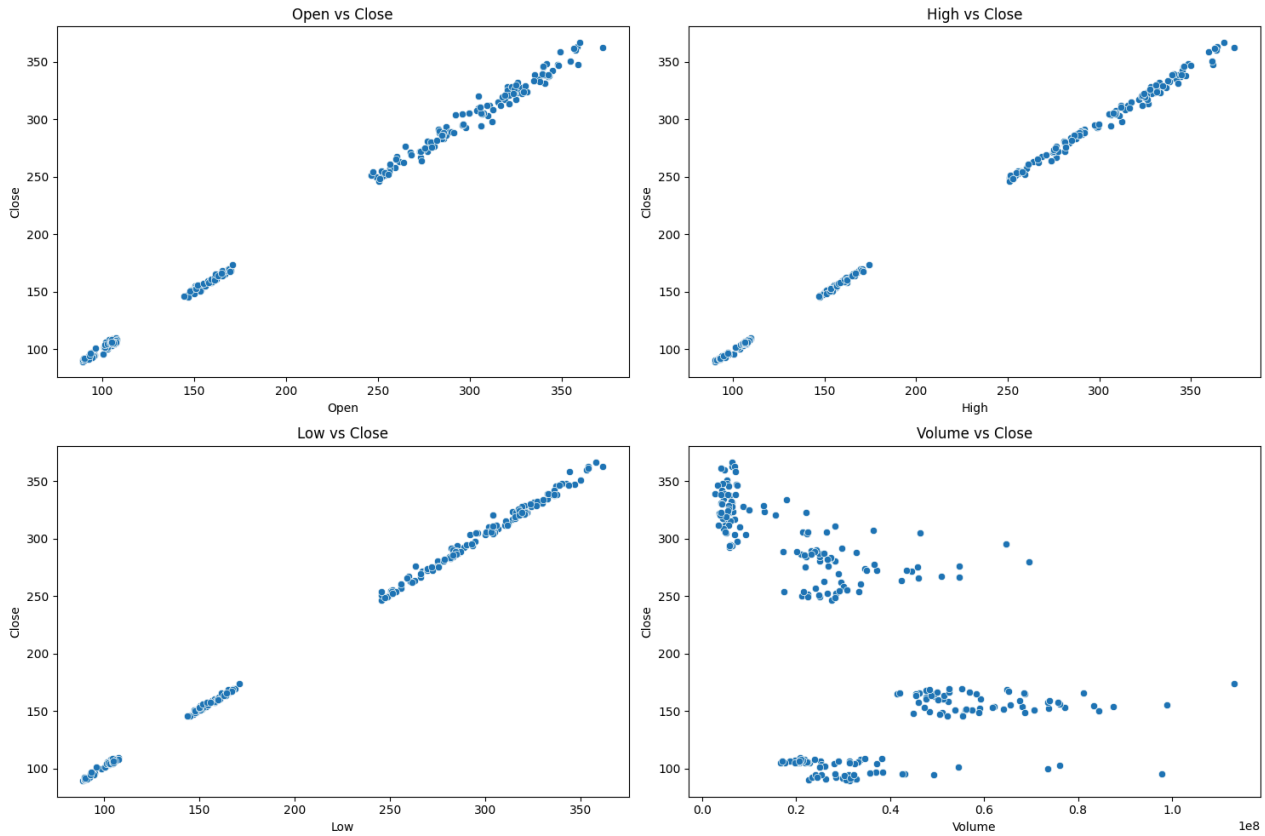
```
plt.figure(figsize=(12, 6))
plt.plot(df['Date'], df['Close'], label='Closing Price')
plt.xlabel('Date')
plt.ylabel('Price')
plt.title('Stock Closing price Over Time')
plt.legend()
plt.grid(True)
plt.show()
```




```
df.describe()
```

	Date	Open	High	Low	Close	Adj Close	Volume
count	248	248.000000	248.000000	248.000000	248.000000	248.000000	2.480000e+02
mean	2023-03-22 22:27:05.806451712	215.252093	217.919662	212.697452	215.381674	215.362697	3.208210e+07
min	2023-02-07 00:00:00	89.540001	90.129997	88.860001	89.349998	89.349998	2.657900e+06
25%	2023-03-01 00:00:00	135.235004	137.440004	134.822495	136.347498	136.347498	1.714180e+07
50%	2023-03-22 12:00:00	208.764999	212.614998	208.184998	209.920006	209.920006	2.734000e+07
75%	2023-04-14 00:00:00	304.177505	307.565002	295.437500	303.942505	303.942505	4.771772e+07
max	2023-05-05 00:00:00	372.410004	373.829987	361.739990	366.829987	366.829987	1.133164e+08
std	NaN	91.691315	92.863023	90.147881	91.461989	91.454750	2.233590e+07

```
features = ['Open', 'High', 'Low', 'Volume']
plt.figure(figsize=(15, 10))
for i, feature in enumerate(features, 1):
  plt.subplot(2, 2, i)
  sns.scatterplot(x=df[feature], y=df['Close'])
  plt.title(f'{feature} vs Close')
  plt.xlabel(feature)
  plt.ylabel( 'Close' )
plt.tight_layout()
plt.show()
```

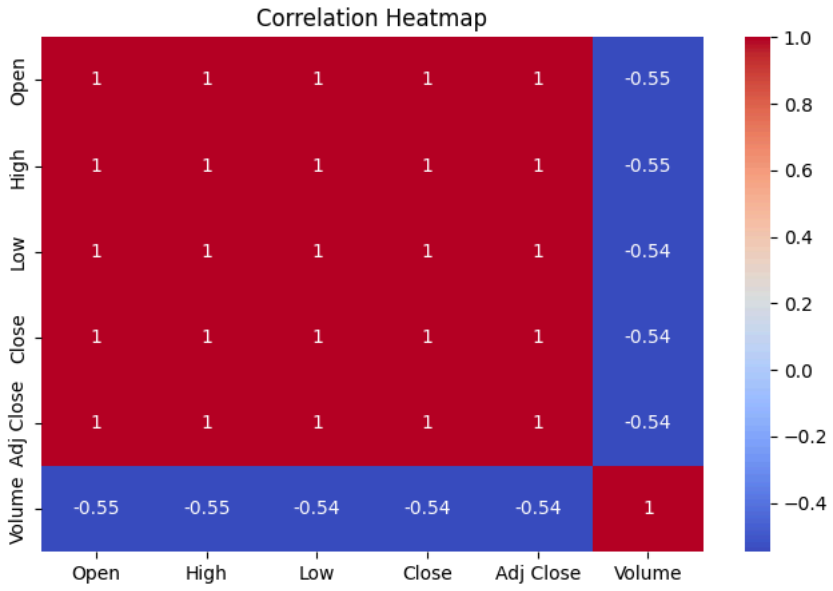


FEATURE SELECTION

```
correlation = df.corr(numeric_only=True)['Close'].sort_values(ascending=False)
print(correlation)
```

```
Close        1.000000
Adj Close    0.999999
Low          0.999663
High         0.999644
Open         0.999176
Volume      -0.544194
Name: Close, dtype: float64
```

```
plt.figure(figsize=(8, 5))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```




```
df['price_Change'] = df['Close'] - df['Open']
```

```
print(df['price_Change'])
```

```
0      4.009995
62     7.029999
186    4.410004
124    4.440002
187   -2.690002
          ...
60     0.899994
123    4.929993
61     2.590012
185   -0.849976
247    0.894997
Name: price_Change, Length: 248, dtype: float64
```

```
df['MovingAverage7'] = df['Close'].rolling(window=7).mean()
df['MovingAverage21'] = df['Close'].rolling(window=21).mean()
```

```
from sklearn.model_selection import train_test_split
Features = ['Open', 'High', 'Low', 'Volume', 'MovingAverage7', 'MovingAverage21', 'price_Change']
target = 'Close'
X = df[Features]
y = df[target]
data = df.dropna()
```

```
X = data[features]
y = data[target]
# SpLit data (e.g., 80% train, 20% test)
split_index = int(len(data) * 0.8)
X_train, X_test = X[:split_index], X[split_index:]
y_train, y_test = y[ : split_index] , y[split_index: ]
```

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
# Fit onLy on training data
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
df. reset_index(inplace=True)
```

```
print(df.columns)
```

```
Index(['index', 'Ticker', 'Date', 'Open', 'High', 'Low', 'Close', 'Adj Close',
       'Volume', 'price_Change', 'MovingAverage7', 'MovingAverage21'],
      dtype='object')
```

```
df['Date'] = pd.to_datetime(df['Date'])
```

MODEL SELECTION

```
from sklearn.linear_model import LinearRegression,Ridge, Lasso
lin_reg = LinearRegression()
lin_reg.fit(X_train_scaled, y_train)
```

```
▾ LinearRegression
LinearRegression()
```

```
y_pred_lr = lin_reg.predict(X_test_scaled)
```

```
ridge_reg = Ridge(alpha=1.0)
ridge_reg .fit (X_train_scaled, y_train )
```

```
▾ Ridge
Ridge()
```

```
y_pred_ridge = ridge_reg . predict (X_test_scaled )
```

```
lasso_reg = Lasso(alpha=0.1)
lasso_reg.fit(X_train_scaled, y_train)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_coordinate_descent.py:695: ConvergenceWarning: Objective did r
  model = cd_fast.enet_coordinate_descent(
▾ Lasso
Lasso(alpha=0.1)
```

```
y_pred_lasso = lasso_reg.predict(X_test_scaled)
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
def evaluate_model(y_true, y_pred, model_name):
  mse = mean_squared_error(y_true, y_pred)
  r2 = r2_score(y_true, y_pred)
  print(f"{model_name} Performance=")
  print(f" MSE= {mse:.2f}")
  print(f" R2 Score= {r2:.4f}\n")
evaluate_model(y_test, y_pred_lr, "Linear Regression")
evaluate_model(y_test, y_pred_ridge, "Ridge Regression")
evaluate_model(y_test, y_pred_lasso, "Lasso Regression")
```

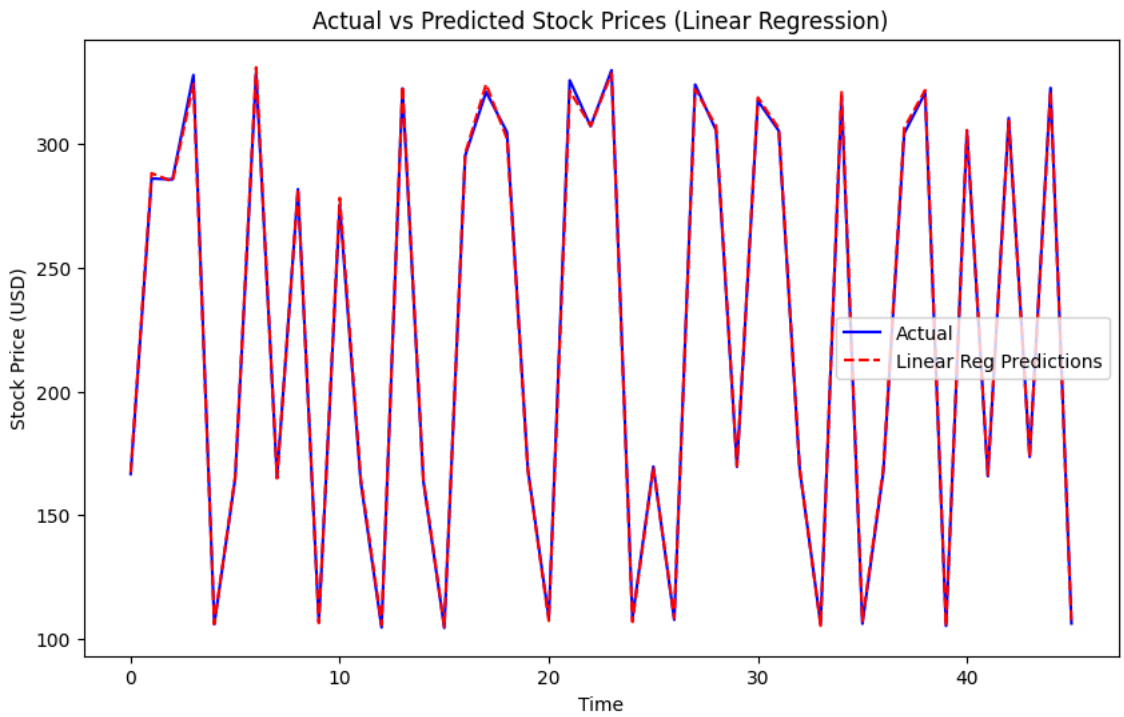
```
Linear Regression Performance=
 MSE= 2.35
 R2 Score= 0.9997

Ridge Regression Performance=
 MSE= 3.69
 R2 Score= 0.9995

Lasso Regression Performance=
 MSE= 4.05
 R2 Score= 0.9995
```

```
plt.figure(figsize=(10,6))
plt.plot(y_test.values, labe1='Actua1', color='blue')
plt.plot(y_pred_lr, label=Linear Reg Predictions, color=red,
plt.title("Actual vs Predicted Stock Prices (Linear Regression)")
plt.xlabe1("Time")
plt.ylabel("stock Price (USD)")
plt.1egend()
plt.show()
line style=--)
```

```
  File "/tmp/ipython-input-326/3847898808.py", line 7
    plt.1egend()
        ^
SyntaxError: invalid decimal literal
```

```
plt.figure(figsize=(10,6))
plt.plot(y_test.values, label='Actual', color='blue')
plt.plot(y_pred_lr, label='Linear Reg Predictions', color='red', linestyle='--')
plt.title("Actual vs Predicted Stock Prices (Linear Regression)")
plt.xlabel("Time")
```

```
plt.ylabel("Stock Price (USD)")
plt.legend()
plt.show()
```



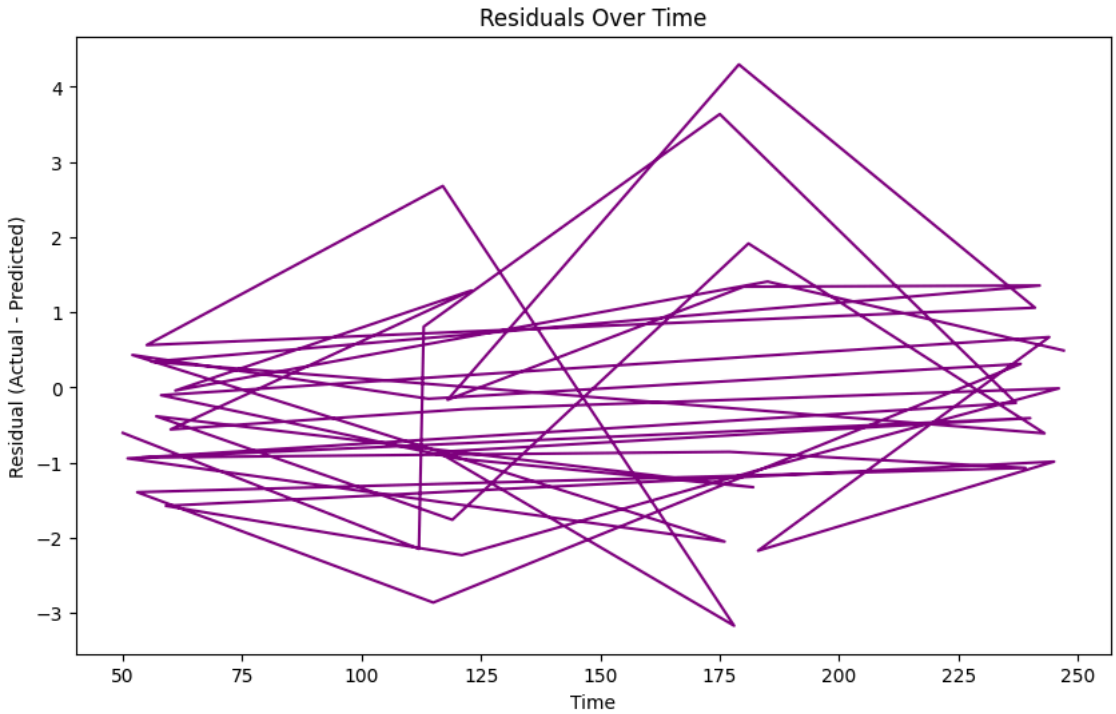
MODEL EVALUATION

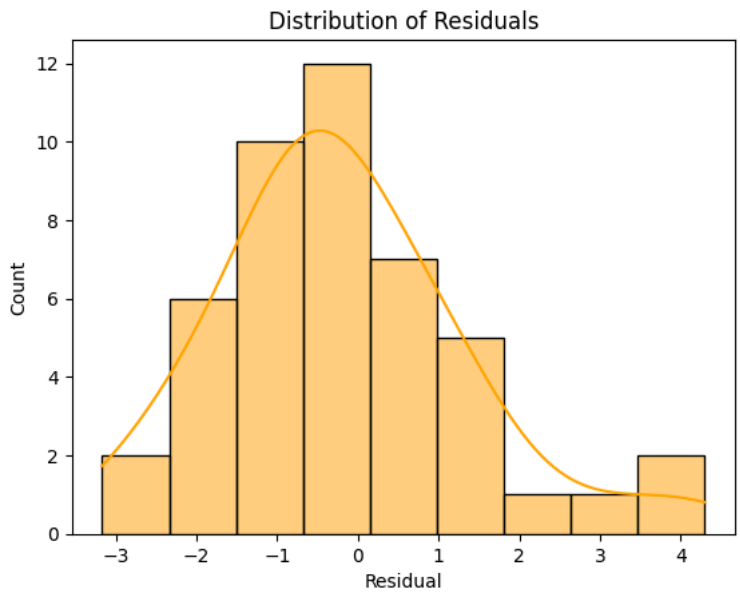
```
y_pred = lin_reg.predict(X_test_scaled)
```

```
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
print("Model Evaluation Metrics-")
print(f"Mean Absolute Error (MAE)= {mae:.2f}")
print(f"Mean Squared Error (MSE)= {mse:.2f}")
print(f"Root Mean Squared Error (RMSE)= {rmse:.2f}")
print(f"R-squared (R2)= {r2:.4f}")
```

```
Model Evaluation Metrics-
Mean Absolute Error (MAE)= 1.18
Mean Squared Error (MSE)= 2.35
Root Mean Squared Error (RMSE)= 1.53
R-squared (R2)= 0.9997
```

```
residuals = y_test - y_pred
plt.figure(figsize=(10,6))
plt.plot(residuals, color='purple' )
plt.title('Residuals Over Time' )
plt.xlabel('Time')
plt.ylabel('Residual (Actual - Predicted)')
plt.show()
sns.histplot(residuals, kde=True, color='orange')
plt.title('Distribution of Residuals')
plt.xlabel('Residual')
plt.show()
```






```
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
# Decision Tree
dt_model = DecisionTreeRegressor(random_state=42)
dt_model.fit(X_train_scaled, y_train)
y_pred_dt = dt_model.predict(X_test_scaled)
```

```
rF_model = RandomForestRegressor(n_estimators=100, random_state=42)
rF_model.fit(X_train_scaled, y_train)
y_pred_rf = rF_model.predict(X_test_scaled)
```

```
# EvaLuate both modeLs
def evaluate(y_true, y_pred, model_name):
    mae = mean_absolute_error(y_true, y_pred)
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_true, y_pred)
    print(f"\n{model_name} Performance-")
    print(f"MAE= {mae: .2f}")
    print(f"MSE= {mse:.2f}")
    print(f"RMSE= {rmse: .2f}")
    print(f"R2= {r2:.4f}")
evaluate(y_test, y_pred_dt, "Decision Tree Regressors")
evaluate(y_test, y_pred_rf, "Random Forest Regressors")
```

```
Decision Tree Regressors Performance-
MAE=  3.90
MSE= 39.67
RMSE=  6.30
R2= 0.9950

Random Forest Regressors Performance-
MAE=  1.92
MSE= 7.05
RMSE=  2.65
R2= 0.9991
```

MODEL TUNING AND OPTIMIZATION

```
from sklearn.model_selection import GridSearchCV
param_grid = {
'n_estimators': [50, 100, 200],
'max_depth': [5, 16, 20, None],
'min_samples_split': [2, 5, 10]
}
```

```
rf= RandomForestRegressor(random_state=42)
```

```
grid_search = GridSearchCV(
estimator=rf,
param_grid=param_grid,
cv=3,
scoring='r2',
n_jobs=-1,
verbose=2
)
```

```
grid_search.fit(X_train_scaled, y_train)
```

```
Fitting 3 folds for each of 36 candidates, totalling 108 fits
```

GridSearchCV

best_estimator_: RandomForestRegressor

RandomForestRegressor